

Bits, Bytes, Ints

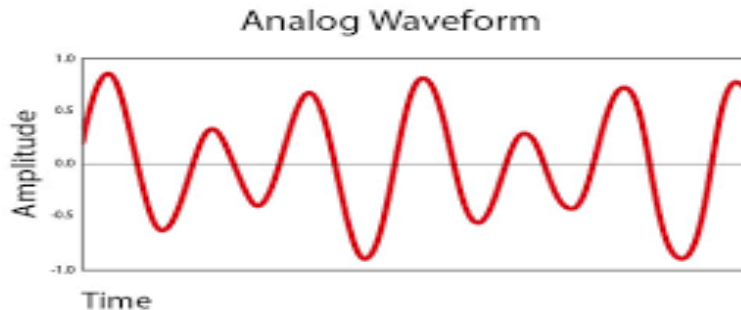
Jinyang Li

Some slides are due to Tiger Wang

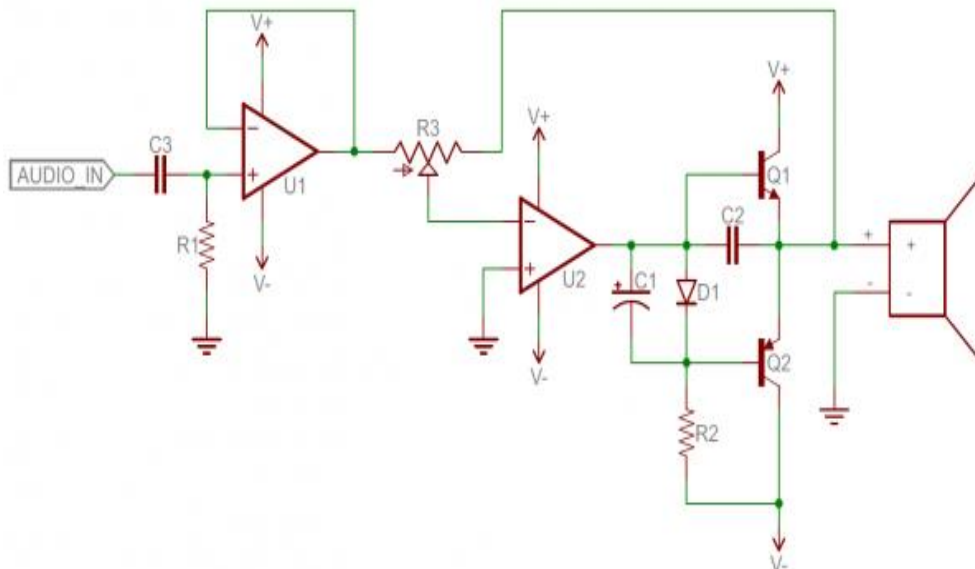
Lesson plan

- How computers represent integers in binary formats
 - Bit, Byte
- How to make binary formats readable to humans
 - Hex notation
- How computers add/subtract integers
- Unsigned vs. signed integer representation

We used to process information as analog signals



Analog signals: smooth and continuous

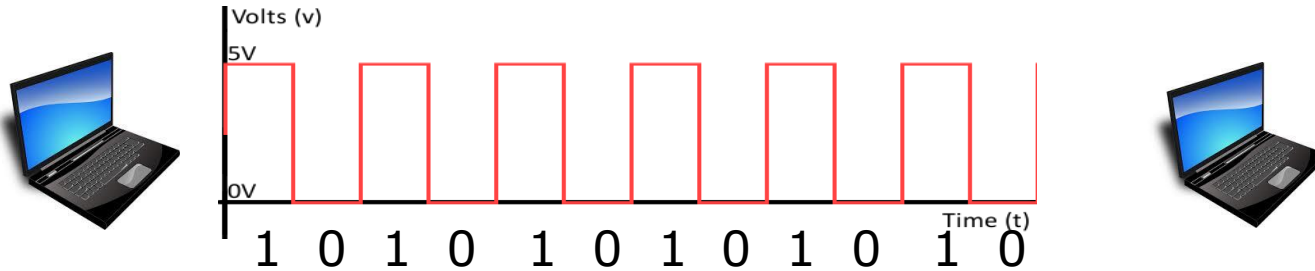


Hard

1. Difficult to design
2. Susceptible to noise

Analog components: resistors, capacitors, inductors, diodes, etc.

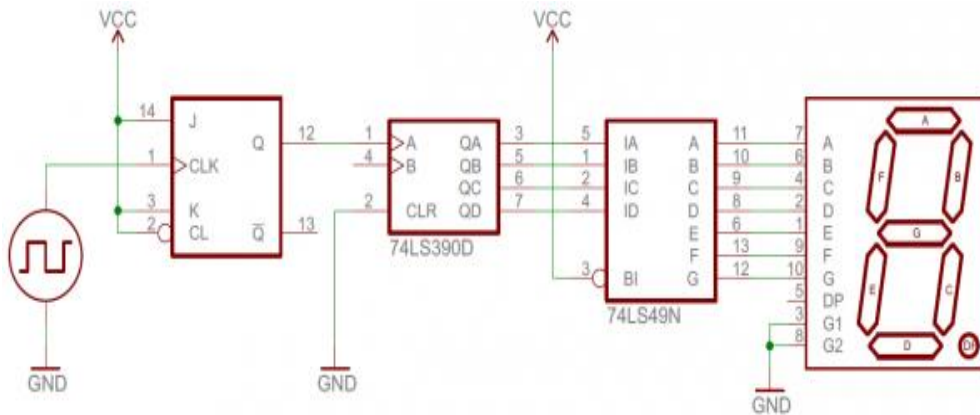
Now, we process information in digital format



Digital signals: discrete (0 or 1)

Easier

- 1. Easier to design**
- 2. Robust to noise**



Digital components: transistors, logic gates ...

Digital information is represented in bits

Bit = Binary digit, 0 or 1

- 1 bit $\rightarrow$ 2 possible values (aka bit patterns)
 - 1 English alphabet $\rightarrow$ 26 “values” (characters)
- Need more values? Use more bits!

Questions

- How many possible 2-bit patterns?

b_0 or b_1 can be either 0 or 1



b_1 b_0

All 2-bit patterns: 00, 01, 10, 11

When poll is active
respond at

PollEv.com
/nyucso

Send **nyucso** to
22333



How many bit patterns?

 0

- How many bit patterns can 8 bits have?

$b_7 b_6 b_5 b_4 b_3 b_2 b_1 b_0$

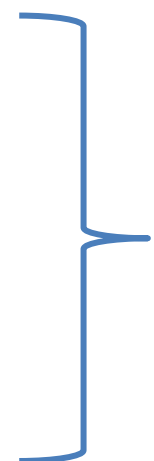
Questions

- How many n-bit patterns?

$b_{n-1} \dots b_1 b_0$

All n-bit patterns:

0...00
0...01
0...10
0...11
...
1...11



2^n

$b_7 \dots b_1 b_0$

 $2^8 = 256$

Digression: All self-respecting CS students should memorize powers of 2

$$2^0 = 1$$

$$2^1 = 2$$

$$2^2 = 4$$

$$2^3 = 8$$

$$2^4 = 16$$

$$2^5 = 32$$

$$2^6 = 64$$

$$2^7 = 128$$

$$2^8 = 256$$

$$2^9 = 512$$

$$2^{10} = 1024$$

Digression: approximations of powers of 2

$$2^{10} = 1024 \approx 10^3 \text{ (Kilo)}$$

$$2^{20} \approx 10^{3*2} = 10^6 \text{ (Mega)}$$

$$2^{30} \approx 10^{3*3} = 10^9 \text{ (Giga)}$$

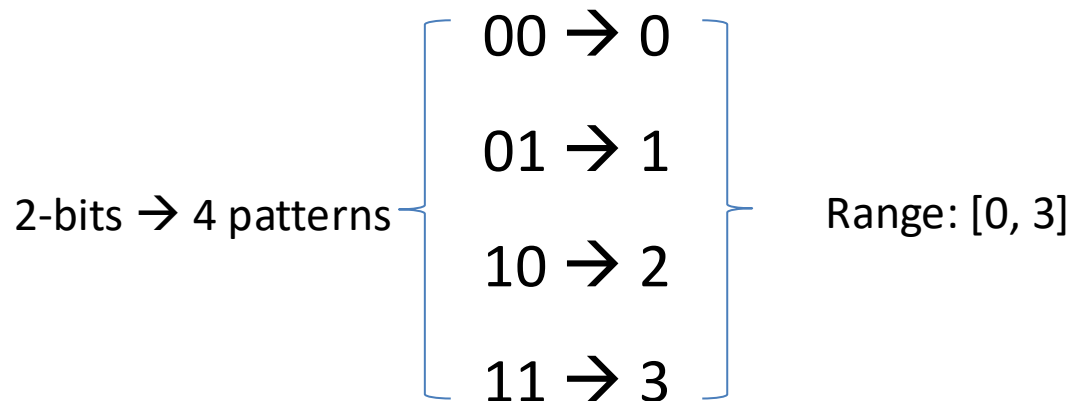
$$2^{40} \approx 10^{3*4} = 10^{12} \text{ (Tera)}$$

$$2^{50} \approx 10^{3*5} = 10^{15} \text{ (Peta)}$$

Represent ~~non-negative~~ unsigned integer

bits: b_1b_0

Goal: map each bit pattern to an integer



Represent unsigned integer

Bit pattern: $b_{n-1}b_{n-2}\dots b_2b_1b_0$

Range: $[0, 2^n - 1]$

Base-2 representation:

$$b_{n-1}b_{n-2}\dots b_2b_1b_0 = \sum_{i=0}^{n-1} b_i * 2^i$$

b_i is bit at i -th position (from right to left, starting at $i=0$)

Examples

Bit pattern: 00000110

Value: $0*2^7 + 0*2^6 + 0*2^5 + 0*2^4 + 0*2^3 + 1*2^2 + 1*2^1 + 0*2^0 = 6$

When poll is active
respond at

PollEv.com
/nyucso

Send **nyucso** to
22333



**What is the decimal value of bit pattern
10000110?**

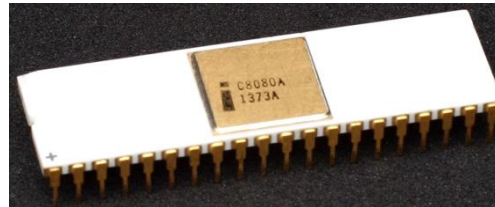
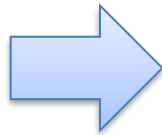
130

0%

Byte



- Byte: a fixed size group of bits
 - The term is coined by Werner Buchholz (IBM).
 - Long long ago, different vendors use different byte sizes
- Now: Byte is 8-bit



IBM System/360, 1964

Introduced 8-bit byte

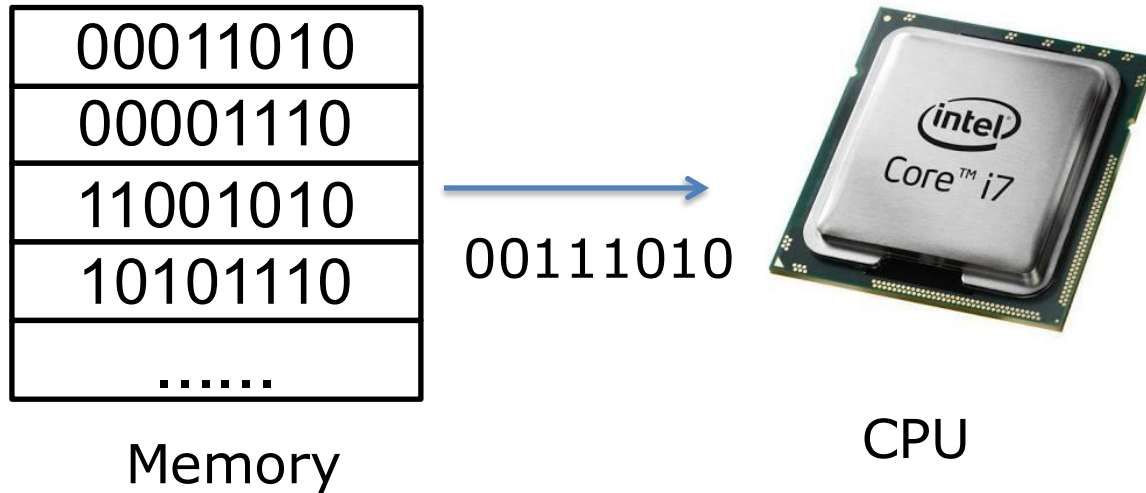
Intel 8080, 1974

Widely adopted

Modern processors

Standardized

Byte



Byte is the smallest unit of information storage, computation and transfer



Integers are represented by 1,2,4, or 8 bytes.

Range of unsigned integer

Size of unsigned int (bytes)	Range
1	
2	
4	
8	

Most and least significant bit/byte

msb: bit position with the largest positional value

lsb: bit position with the smallest positional value

1-byte unsigned int:

10011010

4-byte unsigned int:

01110011 10001101 01010011 11011010



Most significant byte

A light blue rounded rectangle with a pointer at the top left pointing to the first byte (01110011) of the 4-byte integer.



Least significant byte

A light blue rounded rectangle with a pointer at the top left pointing to the last byte (11011010) of the 4-byte integer.

Describing bit patterns in a human-readable way

1-byte int: 10101110

C program:

```
unsigned int a = 0b10101110;
```

If I ask you to type a 4-byte int, ...



Describing a bit pattern: hex notation

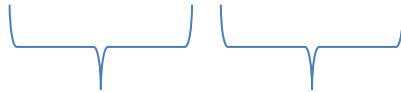
Use one (hex) symbol to represent a group of 4 bits

Binary	Hex	Binary	Hex
0000	0	1000	8
0001	1	1001	9
0010	2	1010	a
0011	3	1011	b
0100	4	1100	c
0101	5	1101	d
0110	6	1110	e
0111	7	1111	f

n-bit binary pattern → ?? hex symbols

Describing a bit pattern: hex notation

10101110



a e

C program:

```
unsigned int a = 0xae;
```

What have we learnt

- How computers represent integers
 - Bit, Byte

Q: What is 10000010 in decimal? A: 130

Q: What's the least significant bit of any even number?

- Hex notation

Q: What is 10001111 in hex? A: 0x8F

Lesson plan

- How computers represent integers
 - Bit, Byte
- Hex notation
- How computers add/subtract integers
- Signed integer representation
 - 2's complement
- A short history of processors:
 - from 8-bit to 64-bit machines
- Byte ordering: big vs. small endian

What's the result?

```
#include <stdio.h>

int main(void) {

    unsigned char a = 0b10000011;
    unsigned char b = 0b10000010;
    unsigned char c = a + b;

    printf("%hhu\n", c);
}
```

<https://godbolt.org/z/Esb1q8KKf>



When poll is active respond
at

[PollEv.com](https://pollev.com/nyucso)
[/nyucso](https://pollev.com/nyucso)



What's the output (unsigned)?

 0

```
#include <stdio.h>
```

```
int main(void)
```

SEE MORE 

How computers add (unsigned integers)

$$\begin{array}{r} 0011 \\ + 0010 \\ \hline \end{array}$$

Result: _ _ _ _ _

How computers add (unsigned integers)

$$\begin{array}{r} 10000011 \\ + 10000010 \\ \hline \end{array}$$

Result: _ _ _ _ _

How to represent signed numbers? signed magnitude

Most significant bit (MSB) represents the sign

$$\begin{array}{l} \text{sign} \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 1_2 \rightarrow 1 \\ \text{magnitude} \\ 1 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 1_2 \rightarrow -1 \end{array}$$

$$\begin{array}{r} 00000001 \\ + 10000001 \\ \hline 10000010 \end{array}$$

-2 ??? 

☹️ Need different h/w for signed vs. unsigned computation

How to represent signed numbers?

Two's complement

Unsigned int

$$00010110 = 0*2^7 + 0*2^6 + 0*2^5 + 1*2^4 + 0*2^3 + 1*2^2 + 1*2^1 + 0*2^0$$

$$10010110 = 1*2^7 + 0*2^6 + 0*2^5 + 1*2^4 + 0*2^3 + 1*2^2 + 1*2^1 + 0*2^0$$

Signed int

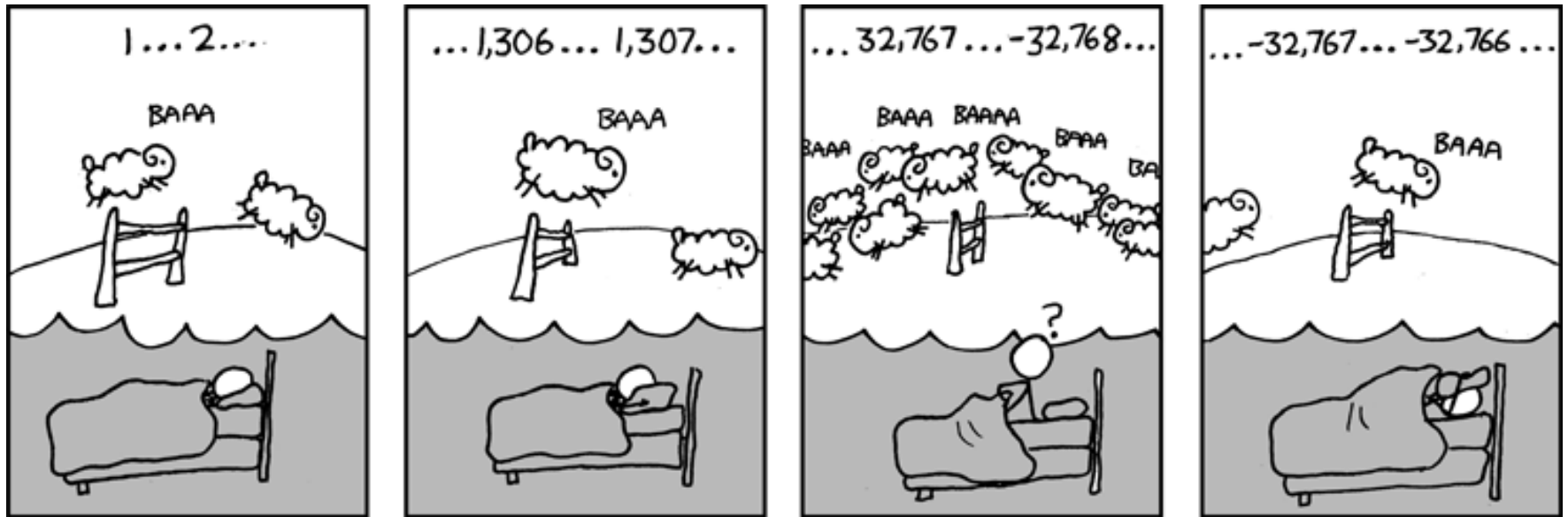
$$00010110 = 0*(-2^7) + 0*2^6 + 0*2^5 + 1*2^4 + 0*2^3 + 1*2^2 + 1*2^1 + 0*2^0$$

$$10010110 = 1*(-2^7) + 0*2^6 + 0*2^5 + 1*2^4 + 0*2^3 + 1*2^2 + 1*2^1 + 0*2^0$$

Two's complement

Bit pattern	Unsigned value	Signed value
00000000	0	0
00000001	1	1
00000010	2	2
...	...	...
01111111	$2^7 - 1 = 127$	$2^7 - 1 = 127$
10000000	$2^7 = 128$	$-2^7 = -128$
10000001	$2^7 + 1 = 129$	$-2^7 + 1 = -127$
10000010	$2^7 + 2 = 130$	$-2^7 + 2 = -126$
...		
11111111	$2^8 - 1 = 255$	$-2^7 + (2^7 - 1) = -1$

Two's complement



Source: xkcd.com

What is the width of the signed integer being enumerated?

Basic facts of 2's complement

Signed int

Size (bytes)	Bit pattern of smallest	Bit pattern of largest	Range
1	0x80	0x7f	$[-2^7, 2^7-1]$
2	0x8000	0x7fff	$[-2^{15}, 2^{15}-1]$
4	0x80000000	0x7fffffff	$[-2^{31}, 2^{31}-1]$
8	0x8000000000000000	0x7fffffffffffffff	$[-2^{63}, 2^{63}-1]$

🤔 Home exercise: make a similar table for unsigned int

- Negative numbers $\leftrightarrow$ MSBit=1
- A sequence of 1's (e.g. 0xff, 0xffffffff) $\leftrightarrow$ -1

Two's complement: 8-bit signed integer

$$00000101 = 0*(-2^7) + 0*2^6 + 0*2^5 + 0*2^4 + 0*2^3 + 1*2^2 + 0*2^1 + 1*2^0 = 5$$

$$11111011 = 1*(-2^7) + 1*2^6 + 1*2^5 + 1*2^4 + 1*2^3 + 0*2^2 + 1*2^1 + 1*2^0 = -5$$

$$00000000 = 0$$

$$11111111 = -1$$

$$10000000 = -2^7 = -128$$

$$01111111 = 2^7 - 1 = 127$$

How do computers add? (2's complement)

$$\begin{array}{r} 00000101 \\ + 11111011 \\ \hline \end{array}$$

Result: _ _ _ _ _

What's the result?

```
#include <stdio.h>

int main(void) {

    char a = 0b01111111;
    char b = 0b00000001;
    char c = a + b;

    printf("%hhd\n", c);
}
```

<https://godbolt.org/z/exvx3eKx7>

Shared activities

Shared with me

☐	Name ^	Shared by
☐	 1. Counselor Case Studies 2023	Emma Wen
☐	 2025 Summer Reading Workshop	Tong Jin

Activities

▼

My activities

Shared activities



How do computers add? (2's complement)

$$\begin{array}{r} 0\ 1\ 1\ 1\ 1\ 1\ 1\ 1 \\ +\ 0\ 0\ 0\ 0\ 0\ 0\ 0\ 1 \\ \hline \end{array}$$

Result: _ _ _ _ _ _ _ _

How do computers subtract?

a: 0 0 0 0 0 1 0 1

b: - 0 0 0 0 0 1 0 1

result: - - - - -

0 0 0 0 0 1 0 1

+ 2's complement of -b

- - - - -

2's complement: find a number's negation

00000101 $\longrightarrow$??
 $(5)_{10}$ $(-5)_{10}$

A useful trick to do negation:

Step-1: flip all bits

00000101 $(5)_{10}$
 $\downarrow$ Step-1: flip bits

Step-2: add 1

11111010
 $\downarrow$ Step-2: +00000001
11111011 $(-5)_{10}$

Why does the negation trick work

$$\vec{b} + (\sim \vec{b}) = 11\dots 11_2 = -1$$

b with bits
flipped

$$-\vec{b} = (\sim \vec{b}) + 1$$

How do computers subtract?

a: 0 0 0 0 0 1 0 1 0 0 0 0 0 1 0 1

b: - 0 0 0 0 0 1 0 1 + 1 1 1 1 1 0 1 0 ~b (flipped b)

result: - - - - - 0 0 0 0 0 0 0 1

 - - - - -

The diagram illustrates two methods for binary subtraction. The first method, labeled 'a' and 'b', shows a direct subtraction: 00000101 minus 00000101. This method is crossed out with a large red X. The second method, labeled 'a' and 'b', shows the addition of the flipped value of b (11111010) plus 1 (00000001). The result of the addition is 00000001. A red arrow points from the text '~b (flipped b)' to the 11111010 value.

Works for both unsigned and signed subtraction!

How do computers add and subtract?

- Same h/w mechanism for signed or unsigned integers
- Subtracting is done by adding negated b
- Different overflow conditions for unsigned/signed
 - Unsigned:
 - Add: carry out of msb
 - Subtract: no carry out of msb (aka borrow out of msb)
 - Signed:
 - Add: operands have same signs, result has opposite sign
 - Subtract: operands have different signs, result has opposite sign from a.



Unsigned addition

$$\begin{array}{r} 00000001 \text{ (1)}_{10} \\ + 00000011 \text{ (3)}_{10} \\ \hline \end{array}$$

$$00000100 \text{ (4)}_{10}$$

$$\begin{array}{r} 00000001 \text{ (1)}_{10} \\ + 10000001 \text{ (129)}_{10} \\ \hline \end{array}$$

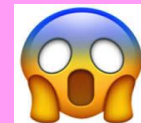
$$10000010 \text{ (130)}_{10}$$

$$\begin{array}{r} 01000001 \text{ (65)}_{10} \\ + 01000000 \text{ (64)}_{10} \\ \hline \end{array}$$

$$10000001 \text{ (129)}_{10}$$

$$\begin{array}{r} 10000001 \text{ (129)}_{10} \\ + 11111110 \text{ (254)}_{10} \\ \hline \end{array}$$

$$01111111 \text{ (127)}_{10}$$



Overflow!



Signed addition

$$\begin{array}{r} 00000001 \text{ (1)}_{10} \\ + 00000011 \text{ (3)}_{10} \\ \hline \end{array}$$

$$00000100 \text{ (4)}_{10}$$

$$\begin{array}{r} 00000001 \text{ (1)}_{10} \\ + 10000001 \text{ (-127)}_{10} \\ \hline \end{array}$$

$$10000010 \text{ (-126)}_{10}$$

This is what 2's complement is designed to accomplish!

$$\begin{array}{r} 01000001 \text{ (65)}_{10} \\ + 01000000 \text{ (64)}_{10} \\ \hline \end{array}$$

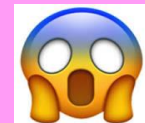
$$10000001 \text{ (-127)}_{10}$$



Overflow!

$$\begin{array}{r} 10000001 \text{ (-127)}_{10} \\ + 11111110 \text{ (-2)}_{10} \\ \hline \end{array}$$

$$01111111 \text{ (127)}_{10}$$



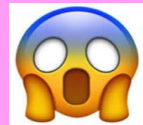
Overflow!



Unsigned subtraction

$$\begin{array}{r} 00000001 \text{ (1)}_{10} \\ - 00000011 \text{ (3)}_{10} \\ \hline \end{array}$$

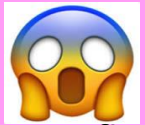
11111110 (254)₁₀



Overflow!

$$\begin{array}{r} 00000001 \text{ (1)}_{10} \\ - 11111111 \text{ (255)}_{10} \\ \hline \end{array}$$

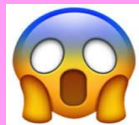
00000010 (2)₁₀



Overflow!

$$\begin{array}{r} 01111111 \text{ (127)}_{10} \\ - 11111110 \text{ (254)}_{10} \\ \hline \end{array}$$

10000001 (129)₁₀



Overflow!

$$\begin{array}{r} 10000000 \text{ (128)}_{10} \\ - 00000001 \text{ (1)}_{10} \\ \hline \end{array}$$

01111111 (127)₁₀



Signed subtraction

$$\begin{array}{r} 00000001 \text{ (1)}_{10} \\ - 00000011 \text{ (3)}_{10} \\ \hline \end{array}$$

$$11111110 \text{ (-2)}_{10}$$

$$\begin{array}{r} 00000001 \text{ (1)}_{10} \\ - 11111111 \text{ (-1)}_{10} \\ \hline \end{array}$$

$$00000010 \text{ (2)}_{10}$$

$$\begin{array}{r} 01111111 \text{ (127)}_{10} \\ - 11111110 \text{ (-2)}_{10} \\ \hline \end{array}$$

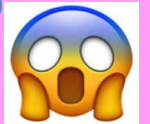
$$10000001 \text{ (-127)}_{10}$$



Overflow!

$$\begin{array}{r} 10000000 \text{ (-128)}_{10} \\ - 00000001 \text{ (1)}_{10} \\ \hline \end{array}$$

$$01111111 \text{ (127)}_{10}$$



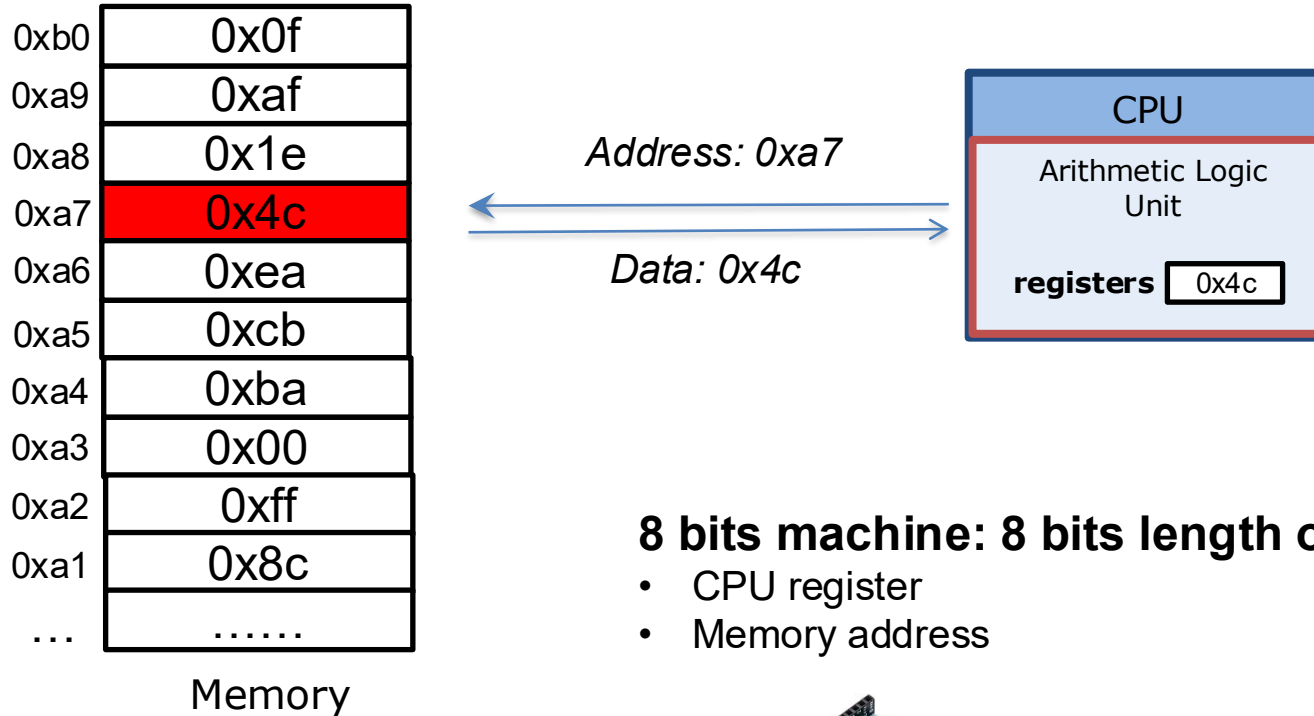
Overflow!

Lesson plan

- How computers represent integers
 - Bit, Byte
- Hex notation
- How computers add/subtract integers
- Signed integer representation
 - 2's complement
- A short history of processors:
 - from 8-bit to 64-bit machines
- Byte ordering: big vs. small endian

THE EVOLUTION OF INTEGER SIZES IN PROCESSORS

8-bit processors: Intel 8080 (1974)



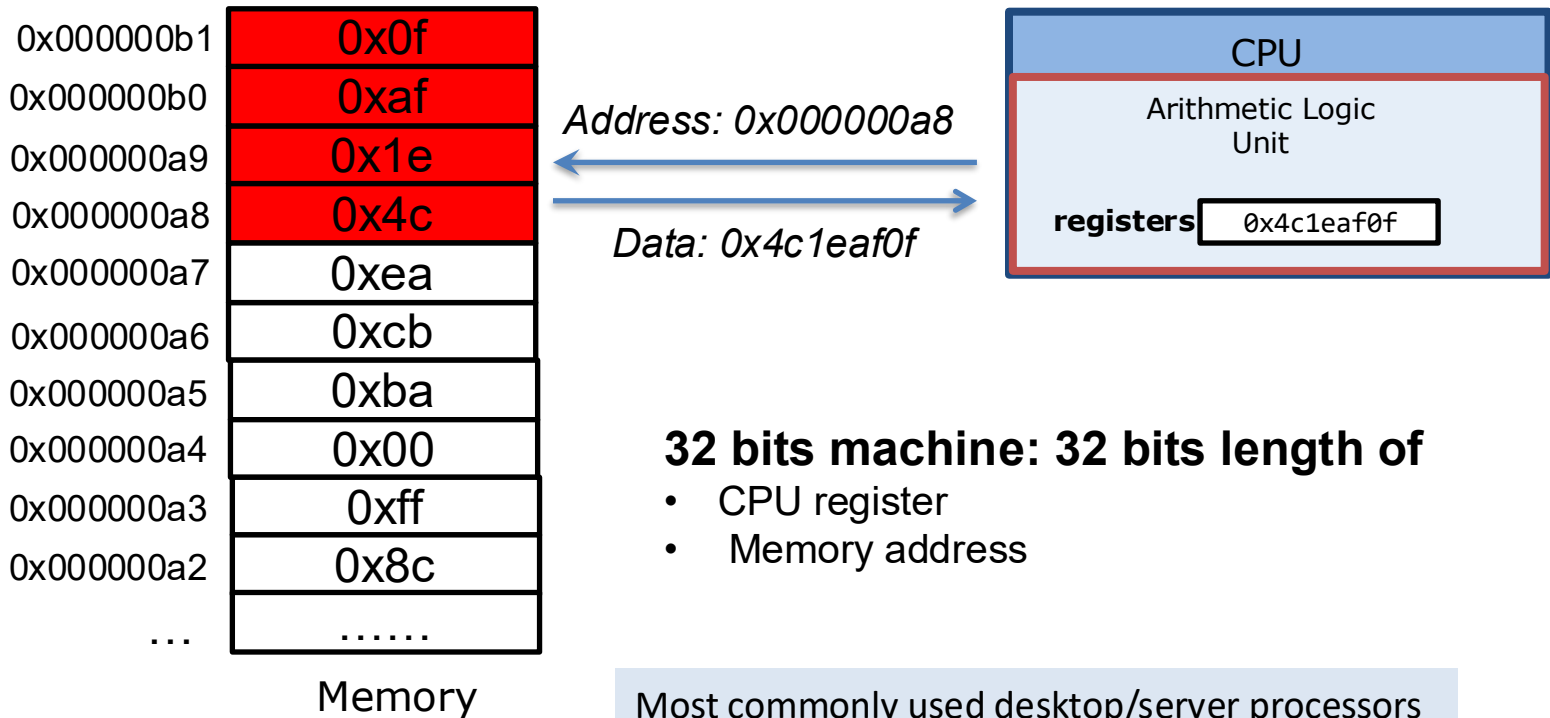
8 bits machine: 8 bits length of

- CPU register
- Memory address



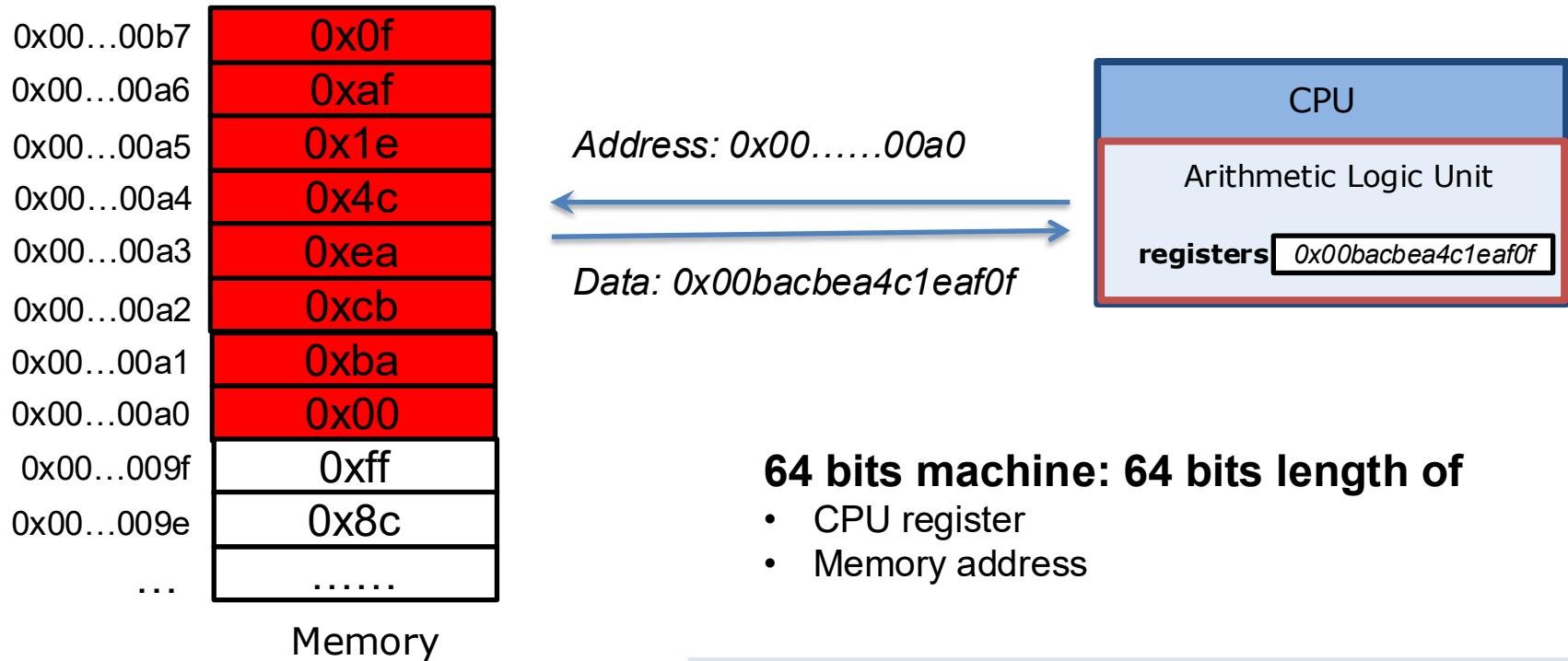
Nowadays: 8-bit processor (microcontroller)
Is used for embedded systems

32-bit processors: Intel 386 (1985)



Most commonly used desktop/server processors in the latest 80s to early 00s

64-bit processors: Intel Pentium 4 (2000)



64 bits machine: 64 bits length of

- CPU register
- Memory address

Nowadays:

- Servers/laptops: Intel/AMD 64-bit x86 processors
- Mobile phones/tablets: 64-bit ARM processors (made by Apple/Qualcomm/Samsung etc)

C's Integer data types on 64-bit machine

	Length	Min	Max
char	1 byte	-2^7	$2^7 - 1$
unsigned char	1 byte	0	$2^8 - 1$
short	2 bytes	-2^{15}	$2^{15} - 1$
unsigned short	2 bytes	0	$2^{16} - 1$
int	4 bytes	-2^{31}	$2^{31} - 1$
unsigned int	4 bytes	0	$2^{32} - 1$
long	8 bytes	-2^{63}	$2^{63} - 1$
unsigned long	8 bytes	0	$2^{64} - 1$

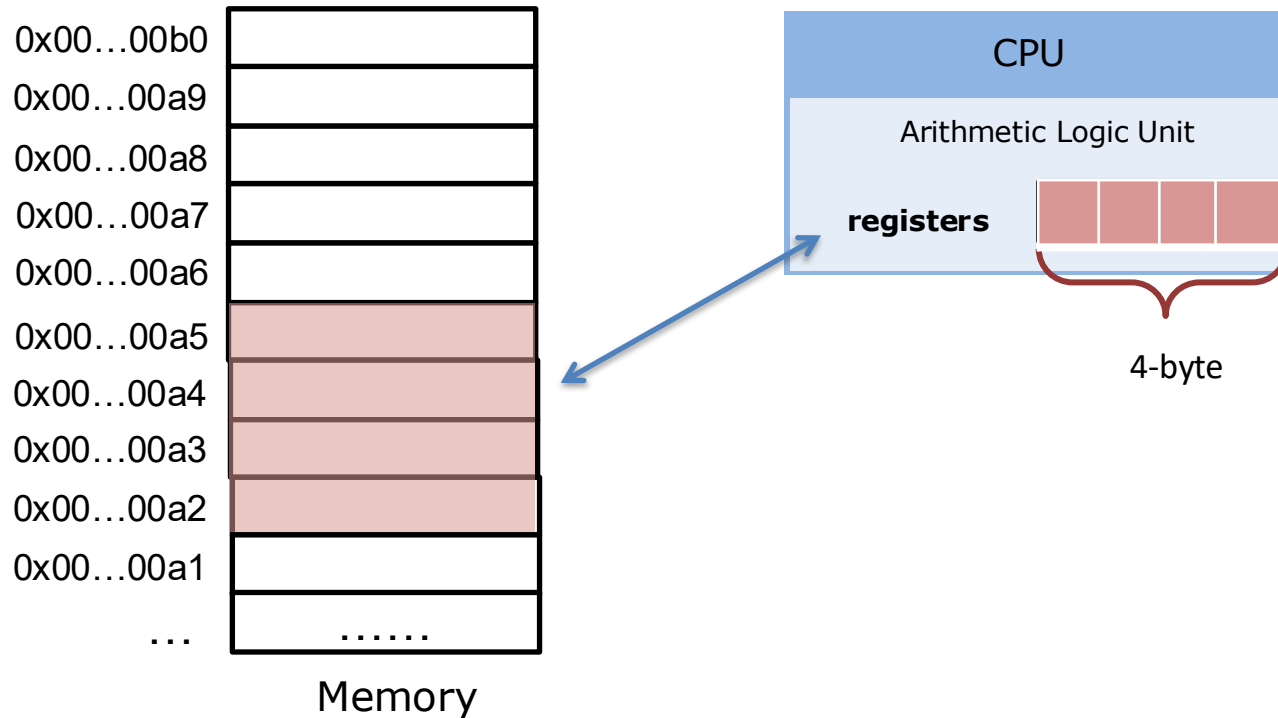
Your first C program

```
#include <stdio.h>

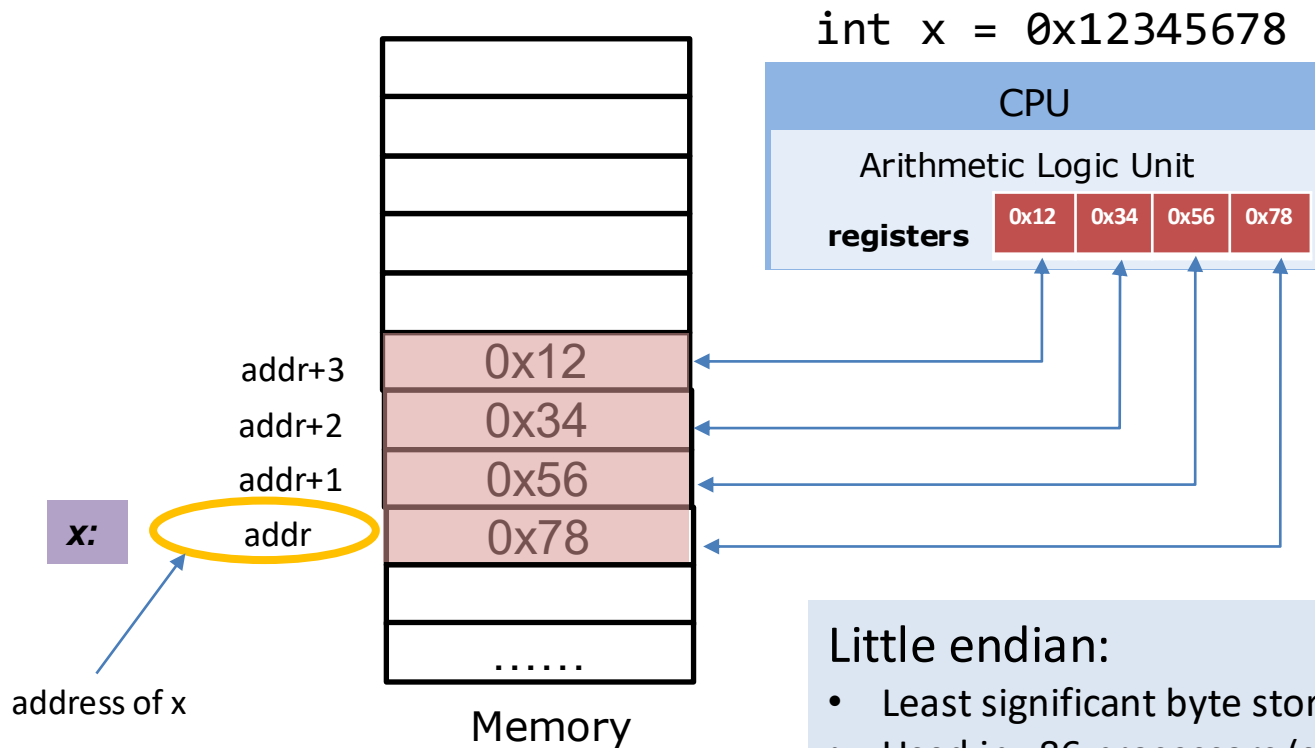
int
main()
{
    char x = -127;
    char y = 0x81;
    char z = x + y;
    printf("hello world sum is %d\n", z);
}
```

\$ gcc helloworld.c		1 0 0 0 0 0 0 1	-127
\$./a.out	+	1 0 0 0 0 0 0 1	-127
		<u>1 0 0 0 0 0 0 1 0</u>	2

Memory layout for multi-byte integers



Memory layout: Little Endian



Advantage of Little Endian

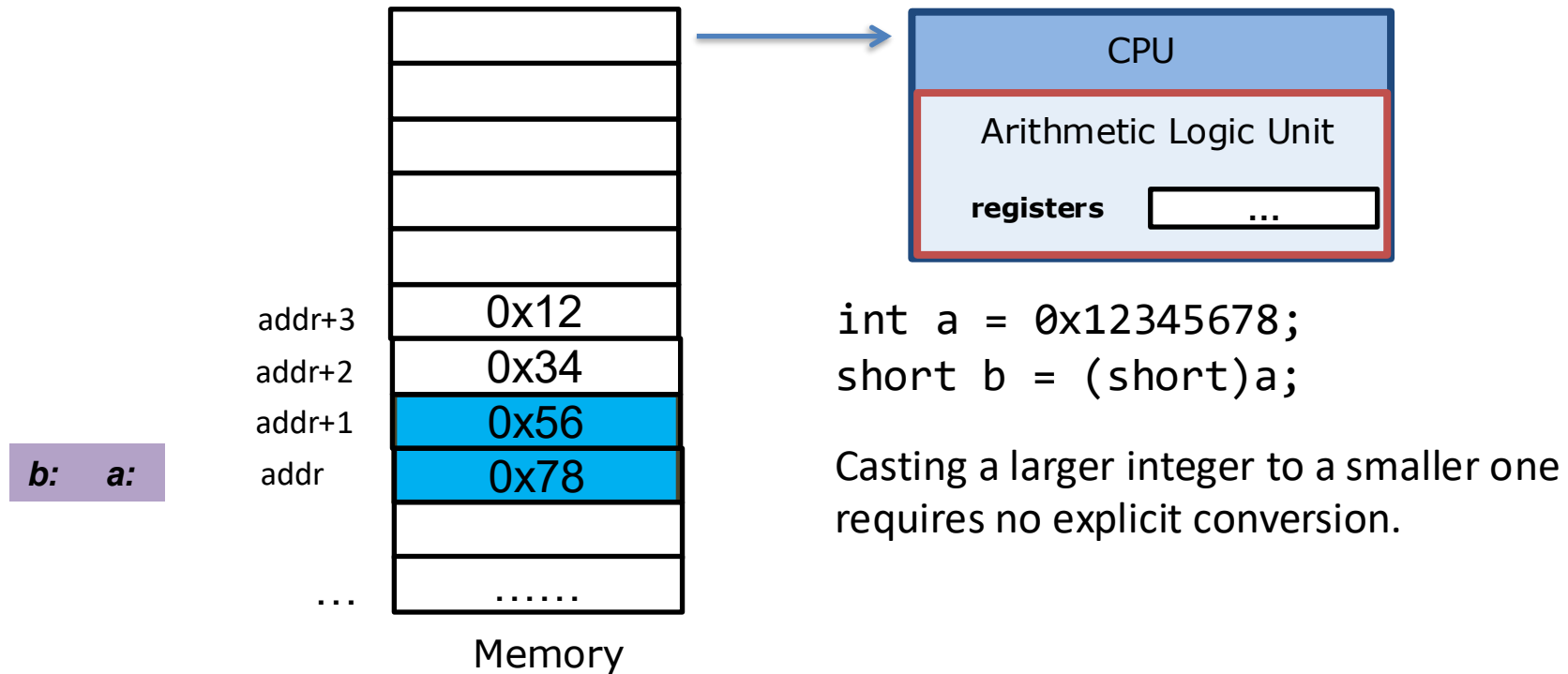
0x12345678
+ 0x12131415


Processor performs calculation
from the least significant bit

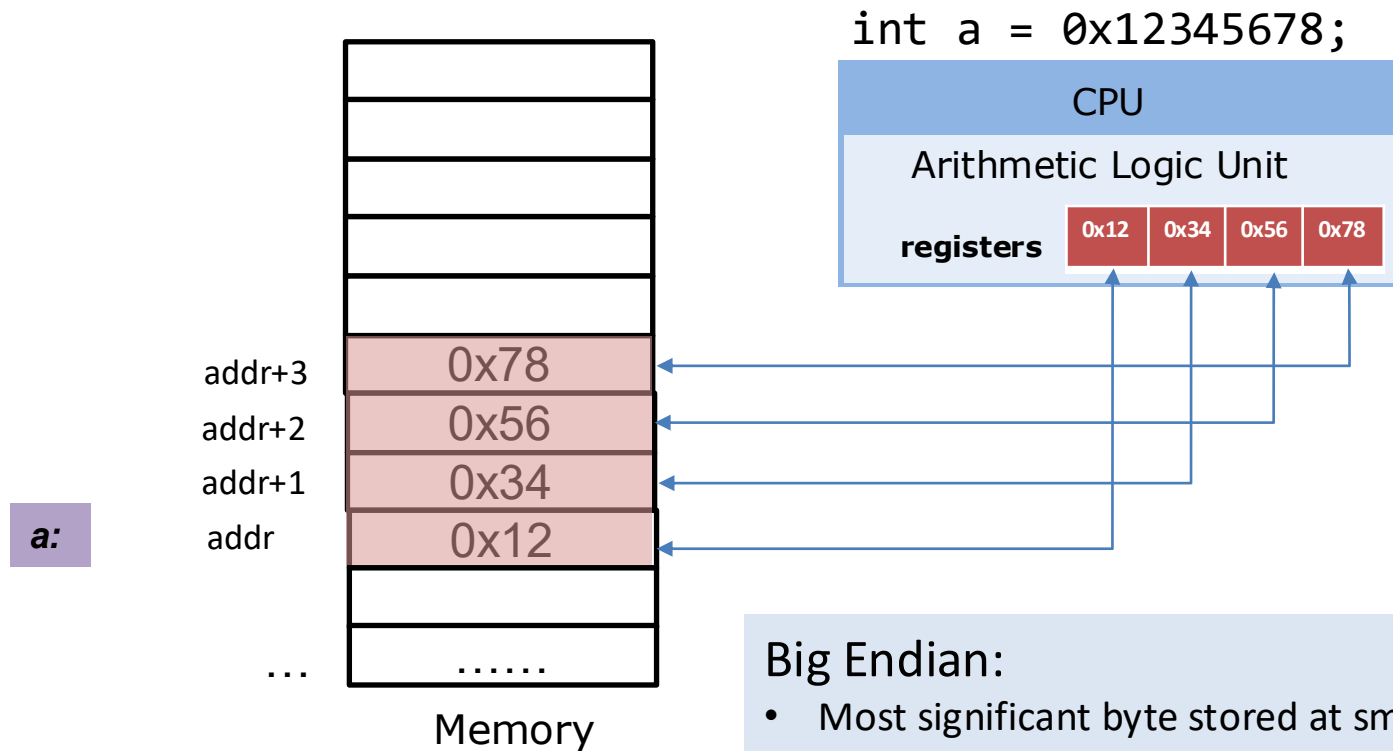


Processor can simultaneously
perform memory transfer and
calculation.

Another advantage of Little Endian



Memory layout: Big Endian



Big Endian:

- Most significant byte stored at smallest address
- ARM processors (phones/tablets) in principle support both endian

Advantages of Big Endian

Quick to test whether the number is positive or negative

- Examine byte stored at the address offset zero.

Big or Little Endian?

```
#include <stdio.h>

int main()
{
    int x = 0x05060708;
    int *p;
    p = &x; //p contains the address of x
    char y = *(char *)p; //y is the 1-byte value pointed to by p
    printf("y=%d\n", y);
}
```

```
$ gcc endian.c
$ ./a.out
```

Summary

- Integer representation
 - Unsigned (base-2)
 - Signed (2's complement)
- Hex notation
- Operations (e.g. add, subtract) on fixed-width integers can cause overflow
- Big vs. little endian